

CSCI 112

Programming with C

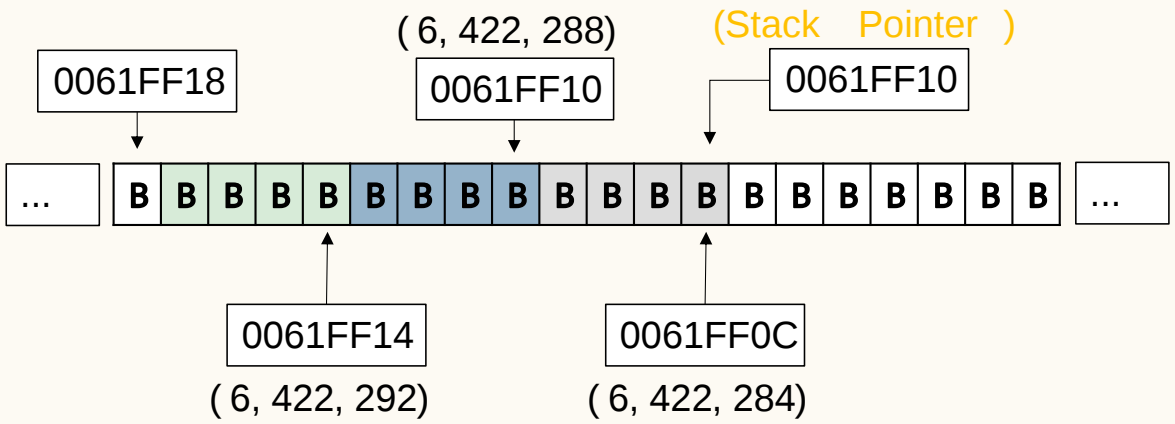
LECTURE: 17

J. L. Pach

OUTLINE:

- Where do variables live? The stack and the heap
- Allocating memory: `malloc()`, `calloc()`, `realloc()`
- Releasing memory: `free()`
- What can go wrong: leaks, dangling pointers, double free
- Defensive habits: checking, nullifying, the `realloc` pattern
- Working with raw memory: `memcpy()`, `memmove()`, `memcmp()`, `memset()`, `memchr()`
- Building a growing buffer; arrays of strings

The stack: local variables 1/4




```c
int x = 0x12345678, y = 0xAABBCCDD, z = 0x11223344;

printf("%-4s equals %d.\n", "x", &x); // Result: "x    equals 6422292."
printf("%-4s equals %d.\n", "y", &y); // Result: "y    equals 6422288."
printf("%-4s equals %d.\n", "z", &z); // Result: "z    equals 6422284."
```

- The compiler, when declaring a variable `x` of type `int`, reserves 4 bytes of memory, which means it decrements the stack pointer (held in the stack pointer register) by 4,

> then writes the contents from the least significant bit to the most significant bit starting from that address.

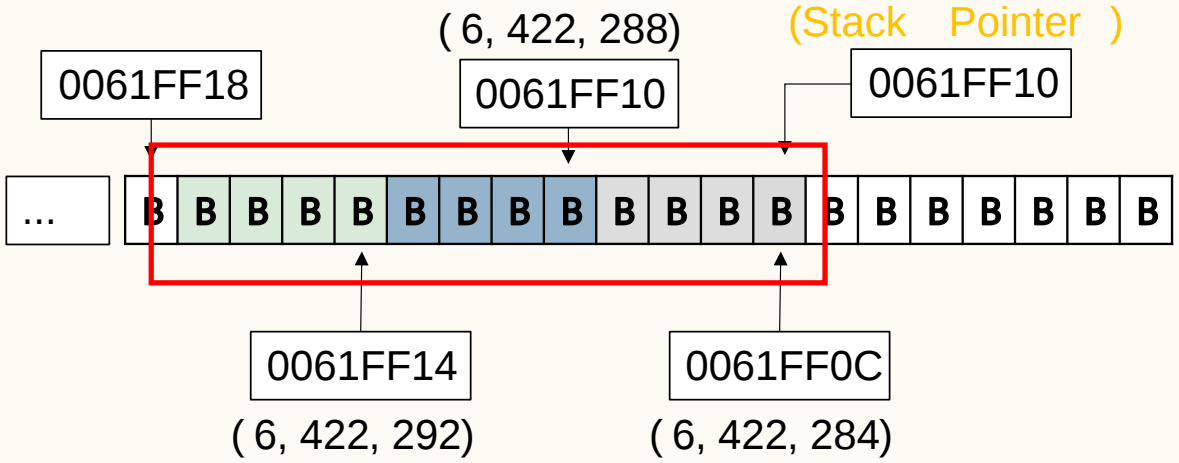
- It then proceeds similarly with the next variable `y` of type `int`, and analogously with the variable `z`.

The stack: local variables 2/4

Address	Value
0061FF18	...
0061FF17	0x12
0061FF16	0x34
0061FF15	0x56
0061FF14	0x78
0061FF13	0xAA
0061FF12	0xBB
0061FF11	0xCC
0061FF10	0xDD
0061FF0F	0x11
0061FF0E	0x22
0061FF0D	0x33
0061FF0C	0x44
0061FF0B	...

SP →

```
int x = 0x12345678, y = 0xAABBCCDD, z = 0x11223344;
```



- Data storage in RAM in Windows systems follows the little-endian standard,
- which means we start counting bits from the least significant bit, i.e., from the 2^0, and at the lowest memory address after reserving space, we write the successive bytes of our multi-byte variable, array, etc.

The stack: local variables 3/4

- It is important to remember that `little-endian` has certain consequences. If we create an entire array of a specific type, its size will correspond to the product of the size of the type in bytes and the number of elements. Memory is first reserved, so the address of the start of the array is calculated first, and then the elements are copied in order from the lowest address (the smallest) to the highest.
- This leads to the consequence that an array, and especially a string, will be written in reverse order compared to a multi-byte number because the string will be stored starting from the lowest byte with the first character to the end, while the number will be stored from the least significant byte to the end.

The stack: local variables 4/4

```c
#define n 6

int i;
char array1[n] = "ABCDE";
int  array2[n] = {97, 98, 99, 100, 101, 102};

printf("The address of array1 = %d\n", array1 );
printf("The address of array2 = %d\n", array2 );
char * ptrC = array1;
printf("array1:\n");

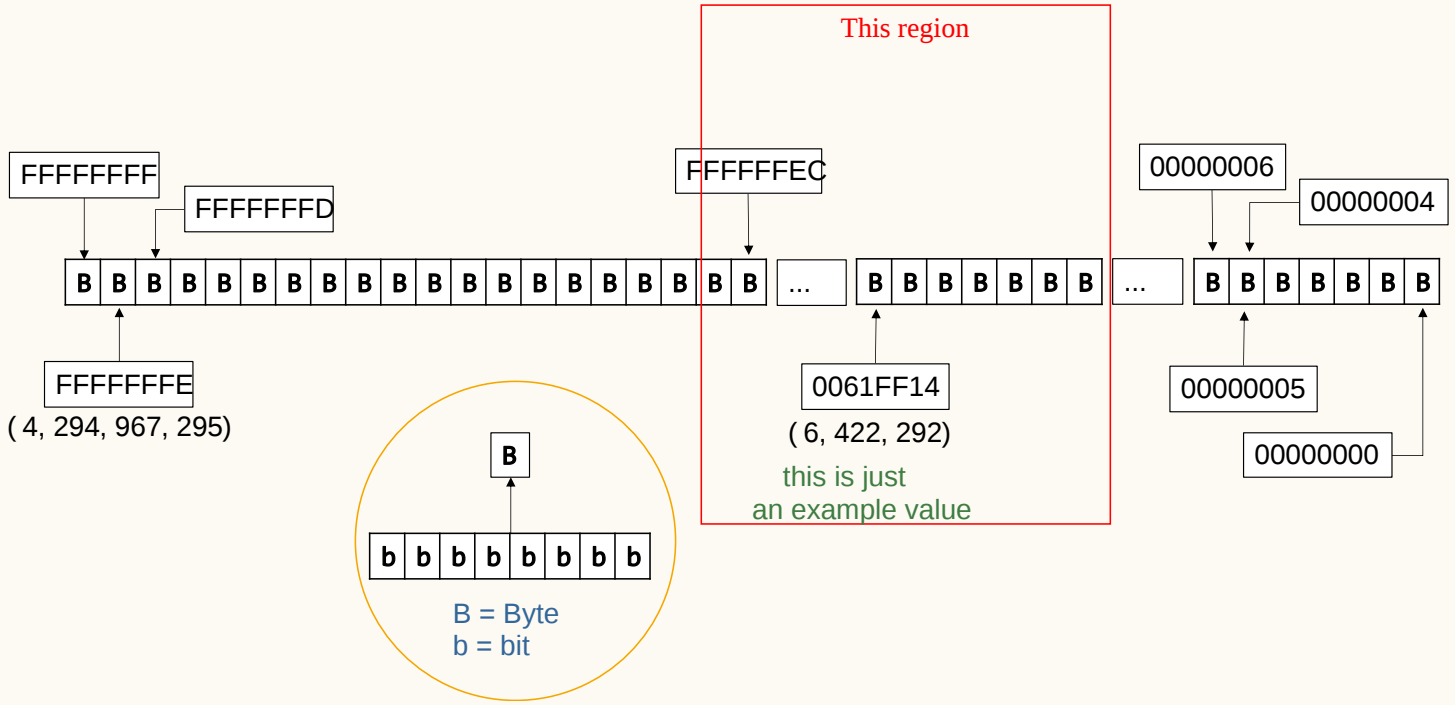
for (i = 0; i < n; ++i)
    printf("Byte address = %d, a hex value: %02x,\
     as a character %c\n", ptrC+i,  ptrC[i], ptrC[i]);
printf("\n");
int * ptrI = array2;
printf("array2:\n");

for (i = 0; i < n; ++i)
    printf("Byte address = %d, a hex value: %02x,\
     as a character %c\n", ptrI+i,  ptrI[i], ptrI[i]);
printf("\n");
```

Result:
The address of array1 = 6487822
The address of array2 = 6487796
array1:
Byte address = 6487822, a hex value: 41, as a character A
Byte address = 6487823, a hex value: 42, as a character B
Byte address = 6487824, a hex value: 43, as a character C
Byte address = 6487825, a hex value: 44, as a character D
Byte address = 6487826, a hex value: 45, as a character E
Byte address = 6487827, a hex value: 00, as a character

array2:
Byte address = 6487796, a hex value: 61, as a character a
Byte address = 6487800, a hex value: 62, as a character b
Byte address = 6487804, a hex value: 63, as a character c
Byte address = 6487808, a hex value: 64, as a character d
Byte address = 6487812, a hex value: 65, as a character e
Byte address = 6487816, a hex value: 66, as a character f

MEMORY



Address	Value
0061FF18	...
0061FF17	0x12
0061FF16	0x34
0061FF15	0x56
0061FF14	0x78
0061FF13	0xAA
0061FF12	0xBB
0061FF11	0xCC
0061FF10	0xDD
0061FF0F	0x11
0061FF0E	0x22
0061FF0D	0x33
0061FF0C	0x44
0061FF0B	...

SP

MEMORY

- The number of memory cells (bytes) we can access is limited by the architecture of the processor. If the processor is 32-bit, it means that it has a memory cell called the stack pointer, which is 32 bits wide. Therefore, the minimum value that can be stored there is 0, and the maximum is `FFFFFFFF`, which equals `4,294,967,295` (4GB). These are the processor's limitations and the theoretical limitations of RAM size.
- The stack is descending and starts at the maximum value, meaning the first memory cell is `FFFFFFFF`, the second is `FFFFFFFE`, and so on. On the stack, when you run a program, there are already reserved fragments of various other programs, including the operating system. The system allocates you free space from which you can use RAM, which means the top of the stack.

DYNAMIC

Memory

INTRODUCTION TO DYNAMIC MEMORY 1/2

- The operating system allocates a specific amount of memory to a program, loads it into `RAM`, and initiates it.
- This allocated memory size is fixed and relatively small. To overcome this limitation, dynamic memory allocation was introduced, which we refer to as the `heap`.
- This means that during the execution of the program (as opposed to when it is loaded and launched), additional `RAM` segments can be allocated in collaboration with the operating system, according to the program's needs and within the available physical memory limits.

INTRODUCTION TO DYNAMIC MEMORY 2/2

In contrast to the program code placed on the stack—which can simply be cleared by adjusting the stack pointer after program execution, preventing memory leaks—dynamically allocated memory requires the program (or programmer) to manually release every byte requested from the operating system. If not freed by the program, these bytes remain locked and unusable until the operating system is restarted, a condition known as a memory leak. This is a common problem. While dynamic memory allocation offers extensive capabilities, it also introduces significant challenges. Many modern applications are developed without a deep understanding of this mechanism, leading to increasing memory usage over time. As memory leaks accumulate, RAM utilization grows until, eventually, the system runs out of memory and the user has to reboot. In theory, the heap is located below the stack, but in practice, the operating system can allocate any free block of RAM.

Stack memory has a short life

When a function returns, its local variables are gone. A pointer to them is **dangling**.

```c
int *make_number(void)
{
    int value = 42;
    return &value;
    /* BUG: value dies when the function returns */
}
int main(void)
{
    int *p = make_number();
    printf("%d\n", *p);
    /* undefined behavior */
    return 0;
}
```

```c
int *make_number(void)
{
    int *value = malloc(sizeof *value);
    if (value != NULL)
    {
        *value = 42;
    }
    return value;
    /* the CALLER now owns the block and must free() it */
}
```

> The compiler usually warns: *"function returns address of local variable"*. **Read warnings.**

> **The fix**: if data must outlive the function, put it on the **heap** (or let the caller provide the storage).

malloc() & free() – Allocating/Deallocating a block of memory

```c
int stackVariable = 5; // Declare a variable on the stack
printf("Address of stackVariable: %p\n", &stackVariable);

// Allocate memory on the heap for an integer
int *heapValue = (int*) malloc( sizeof(int) * 1  );
// Size multiplied by the number of elements
//Since the malloc function returns a void pointer,
//it is recommended to explicitly cast it to the desired type,
//such as int*, to avoid ambiguity.

if (heapValue)  //(heapValue != NULL)
{
  *heapValue = 5; // Assign a value to the allocated memory
  printf("Address of heapValue: %p\n", heapValue);
  printf("The value of dynamicVariable is = %d\n", *heapValue);

  free(heapValue); // Deallocate the memory - so important!
}
else
  printf("Memory allocation failed!\n");
```

```
Result:
Address of stackVariable: 0062ff14
Address of heapValue: 00c41808
The value of dynamicVariable is = 5
```

- To use dynamic memory allocation functions like malloc, you need to include the `stdlib.h` library.
- Malloc and free are always used together.
- The malloc function does not initialize the values of the allocated memory block; the values remain as they were previously in that memory area.
- Malloc expects only one argument, the number of bytes to be allocated.

`malloc()` – allocating a block of memory

- `void *malloc(int bytes);`
- Declared in `<stdlib.h>`.
- Takes **one** argument: the number of **bytes**.
- Returns a pointer to the new block, or `NULL` if the allocation failed.
- The contents are **uninitialized** (indeterminate). Reading them before writing is undefined behavior.
- Every successful `malloc()` must be matched by **exactly one** `free()`.

```c
int *p = malloc(sizeof *p);        /* room for one int */
if (p == NULL)
{
   /* handle the error: return, report, clean up ... */
}
```



THREE `MALLOC()` HABITS

1. **Always check for** `NULL` before you use the block.
2. **Use** `sizeof *pointer` or `sizeof(int)`:

```c
int    *a = malloc(10 * sizeof *a);     /* still correct if you change int to long */
double *b = malloc(n  * sizeof(double));
```



3. **Do not have to cast the result in C.**

```c
int *p = (int *)malloc(sizeof *p);
int *q = malloc(sizeof *q);
```



⚒ Never write the size as a number (`malloc(40)`). Use `count * sizeof(type)`.

STACK AND HEAP TOGETHER



- The **pointer** `heapValue` is an ordinary variable on the stack.
- The **block** it points to is on the heap.
- When the function returns, the pointer disappears automatically. The block does **not**. If you did not `free()` it first and you kept no other pointer to it, the block is **leaked**.

malloc() – an array 1/2

The number of elements may be decided at **run time**.

```c
int m = 3;
int *heapArray = malloc(m * sizeof(int) );
if (heapArray != NULL)
{
  for (int i = 0; i < m; i++)
  {
    heapArray[i] = (int)(i * 10);
    printf("Address of heapArray[%u]: %p\n",\
     i, (void *)&heapArray[i]);
  }
  free(heapArray);
}
```

```
Result:
Address of heapArray[0]: 00650B30
Address of heapArray[1]: 00650B34
Address of heapArray[2]: 00650B38
```

> A stack array `int a[n];` with a variable `n` is a *variable length array* (VLA): it lives on the stack, cannot be resized, and a large `n` can crash the program. Use the heap for large or growing data.

- Malloc does not require static values for allocation.
- Malloc expects only one argument, the number of bytes to be allocated.

```c
const int n = 3, int m = 3, int stackArray[n];

printf("Address of stackArray[0]:\t %.8d\n", &stackArray[0]);  //Result: 'Address of stackArray[0]:       06487768'
printf("Address of stackArray[1]:\t %.8d\n", &stackArray[1]);  //Result: 'Address of stackArray[1]:       06487772'
printf("Address of stackArray[2]:\t %.8d\n\n", &stackArray[2]);//Result: 'Address of stackArray[2]:       06487776'

int *heapArray = (int*) malloc( sizeof(int) * m  );

if (heapArray)
{
  printf("Address of heapArray[0]:\t %.8d\n", &heapArray[0]);  //Result: 'Address of heapArray[0]:       06559344'
  printf("Address of heapArray[1]:\t %.8d\n", &heapArray[1]);  //Result: 'Address of heapArray[1]:       06559348'
  printf("Address of heapArray[2]:\t %.8d\n\n", &heapArray[2]);//Result: 'Address of heapArray[2]:       06559352'

  free(heapArray); // so important!
}
else
  printf("Memory allocation failed!\n");
```

SUMMARY

- When a process is allocated memory for itself, it is copied into RAM, and the stack is placed at the end of this allocated block. Dynamic memory—the heap—is located "outside" of this process's memory space. The operating system assigns an additional block of memory for `malloc()`, which is why addresses in the heap can be larger than those within the process's memory.
- When a program that utilizes dynamically allocated memory crashes and fails to deallocate the memory it has requested, this memory may remain inaccessible until the system is rebooted. While operating systems have mechanisms in place to reclaim such memory, these mechanisms vary significantly between different systems. This phenomenon, where memory is unintentionally retained by a program, is known as a memory leak.

calloc() function 1/2

- Allocates `count * size` bytes and sets **every byte to zero**.
- Takes **two** arguments (number of elements, size of one element).

```c
int n = 3, int *heapArray1 = (int*) calloc( n, sizeof(int)   );
int *heapArray2 = (int*) malloc( sizeof(int) * n  );
if (heapArray1){
  printf("The value of heapArray1[0]:\t %.8d\n", heapArray1[0]);  //Result: `The value of heapArray1[0]:      00000000`
  printf("The value of heapArray1[1]:\t %.8d\n", heapArray1[1]);  //Result: `The value of heapArray1[1]:      00000000`
  printf("The value of heapArray1[2]:\t %.8d\n\n", heapArray1[2]);//Result: `The value of heapArray1[2]:      00000000`
  free(heapArray1); // so important!
}
if (heapArray2)
{
  printf("The value of heapArray2[0]:\t %.8d\n", heapArray2[0]);  //Result: `The value of heapArray2[0]:      -1163005939`
  printf("The value of heapArray2[1]:\t %.8d\n", heapArray2[1]);  //Result: `The value of heapArray2[1]:      -1163005939`
  printf("The value of heapArray2[2]:\t %.8d\n\n", heapArray2[2]);//Result: `The value of heapArray2[2]:      -1163005939`
  free(heapArray2); // so important!
}
```

CALLOC() FUNCTION 2/2

- `calloc()` is similar to `malloc()` but additionally initializes the allocated memory to zero.
- `calloc()` is slower than `malloc()` due to this extra operation. However, `calloc()` is more efficient when you need an array filled with zeros.
- Both `malloc()` and `calloc()` use the free function to deallocate memory.
- It checks the multiplication for **overflow** and returns `NULL` if it would not fit. `malloc(count * size)` can silently wrap around and give you a block that is too small.
- Usually fine to use whenever you need zeros; it is not meaningfully "slower" in everyday code.

realloc() function

```c
int n = 10; int *heapArray1 = (int*) malloc(n * sizeof(int));
// ...
// After some time, we want to reduce the size of the array to 5 elements
int *newHeapArray1 = (int*)realloc(heapArray1, 5 * sizeof(int));
if (newHeapArray1 == NULL)
{
  printf("Handle error: allocation failed\n");
  free(heapArray1);
}
else
{
  printf("Successful allocation, update the pointer\n");
  heapArray1 = newHeapArray1;
}
//...
if(heapArray1)
  free(heapArray1);
// Result: Successful allocation, update the pointer
```

- **Data loss**: When reducing the size of a block, data located outside the new, smaller block will be lost.
- **Data movement**: `realloc()` may move the entire memory block to a new location, so always update the pointer.
- **Freeing memory**: If `realloc()` is successful, the original block will be automatically freed.
- **Syntax**: `void *realloc(void *ptr, int new_size);`

What does `realloc()` return?

Situation	Result
Success	Returns a pointer to the resized block. It may be the **same** address or a **new** one. Old contents are kept (up to the smaller of the two sizes).
Block **moved**	The old block is freed for you. The **old pointer is now invalid** – use the returned one.
Block **shrunk**	Data beyond the new size is lost.
Failure	Returns `NULL`. The original block is **still valid and unchanged**.

> New bytes added by growing are **uninitialized**. Zero them yourself (`memset`) if you need zeros.

realloc() – the safe pattern

```c
/* WRONG: if realloc fails,
p becomes NULL and the old block is LEAKED */
p = realloc(p, new_bytes);
```



```c
/* RIGHT: keep the old pointer until you
know it worked */
int *tmp = realloc(p, new_count * sizeof *p);
if (tmp == NULL)
{
    free(p);     /* or keep using p - it is still valid */
    return false;
}
p = tmp;         /* only now replace the pointer */
```



```c
// Complete example – shrinking an array:
int n = 10;
int *heapArray = malloc(n * sizeof *heapArray);
/* ... after some time we need only 5 elements ... */
int *smaller = realloc(heapArray, 5 * sizeof *smaller);
if (smaller == NULL)
{
    printf("Handle error: reallocation failed\n");
}
else
{
    printf("Successful reallocation, update the pointer\n");
    heapArray = smaller;
}
free(heapArray);
```



`free()` – giving memory back

```c
void free(void *ptr);
```



- `ptr` must be exactly what `malloc` / `calloc` / `realloc` returned – or `NULL`.
- `free(NULL)` is **allowed** and does nothing.
- You may free a block **once**. Freeing it again is a **double free**.
- After `free(p)` the memory is gone, but the **variable** `p` **still holds the old address**. Nothing changes it for you.

Do **not** free:

```c
int x = 5;
free(&x);              /* BUG: stack memory */

int *p = malloc(10 * sizeof *p);
free(p + 3);           /* BUG: not the start of the block */
```



Dangling pointers 1/2

A **dangling pointer** holds the address of memory that has already been freed.

```c
int *p = malloc(sizeof *p);
*p = 7;
free(p);

*p = 8;            /* BUG: use after free   (undefined behavior)        */
printf("%d\n", *p); /* BUG: use after free   (may print 7, 8, garbage...)  */
free(p);           /* BUG: double free      (often crashes)             */
```



⚒ The worst part: the program often **appears to work**. The freed block is still mapped, so you read old data – until the allocator reuses it for something else, and then your write corrupts an unrelated object.

DANGLING POINTERS 2/2

- Common ways to create one:
  - using a pointer after `free()`
  - returning the address of a local variable
  - keeping a copy of a pointer (an *alias*) after the block was freed or moved by `realloc()`

NULLIFY AFTER `FREE()` 1/2

Setting the pointer to `NULL` turns a silent corruption into something you can test and that cannot be freed twice:

```c
free(p);
p = NULL;          /* nullify */

free(p);           /* harmless: free(NULL) does nothing */
if (p != NULL)     /* you can now ask "is this pointer usable?" */
{
    *p = 1;
}
```



Nullify after `free()` 2/2

To nullify **inside a helper function** you need the **address of the pointer** (a pointer to a pointer). Otherwise the function would only change its own copy.

```c
free(p);
p = NULL;
free(p);
if (p != NULL)
{
    *p = 1;
}
```



```c
void safe_free(void **ptr)
{
    if (ptr != NULL)
    {
        free(*ptr);
        *ptr = NULL;
        /* changes the CALLER's variable */
    }
}

safe_free((void **)&p);
```



Nullifying protects only the pointer you nullify. Other copies of the same address (aliases) still dangle.

Memory leaks

A **memory leak**: a block was allocated, you no longer have any pointer to it, and it was never freed – so it can never be freed.

```c
void leak_1(void){
    int *p = malloc(100 * sizeof *p);
    (void)p;
    /* forgot free(p) - the block is lost on return */
}
void leak_2(void){
    int *p = malloc(sizeof *p);
    p = malloc(sizeof *p);
    /* the first block is lost: no pointer to it */
    free(p);
}
int leak_3(int fail){
    int *p = malloc(100 * sizeof *p);
    if (fail)
      return -1;  /* early return skips free(p) */
    free(p);
    return 0;
}
```

- When the program **ends** (even by crashing), the operating system takes back all its memory. Leaks hurt **while the program is running**: servers, games and apps running for days slowly eat all available memory.
- Rule of thumb: for every `malloc` ask *"which line frees this, on every path out of the function?"*

Cleaning up when a later step fails

```c
typedef struct
{
    unsigned char *data;
    int size;
    int capacity;
} ByteBuffer;
ByteBuffer *buffer_create(int capacity)
{
    ByteBuffer *buf = malloc(sizeof *buf);
    /* allocation 1 */
    if (buf == NULL)
    {
        return NULL;
    }
    buf->data = malloc(capacity);
    /* allocation 2 */
    if (buf->data == NULL)
    {
        free(buf);  /* undo allocation 1! */
        return NULL;
    }
    buf->size = 0;
    buf->capacity = capacity;
    return buf;
}
```

- An object made of **two allocations** must not leak the first one when the second fails.
- Destroying it is the same in reverse: free the **inside** (`data`) first, then the **struct**. If you free the struct first, you lose the pointer to `data`.

Memory fragmentation

Memory fragmentation is a problem that can occur when memory is repeatedly allocated and deallocated using functions like malloc and free. Imagine RAM memory as a long tape, on which pieces of memory are allocated. Over time, as we free some pieces, empty spaces are created between the occupied fragments. These free spaces may be too small to accommodate new, larger memory blocks, even if the total amount of free memory is sufficient. This phenomenon is called fragmentation.

Types of fragmentation:

- External fragmentation:
  - Occurs when there are free spaces between occupied memory blocks that are too small to satisfy new allocation requests.
- Internal fragmentation:
  - Occurs when an allocated memory block is larger than actually needed, leading to wasted memory.

CONSEQUENCES OF FRAGMENTATION

- Memory leaks:
  - If memory is not managed carefully, fragmentation can lead to memory leaks, which is a situation where a program no longer frees unnecessary memory, which over time can lead to application crashes.
- Performance degradation:
  - Fragmentation can slow down program execution, as the operating system has to search longer for suitable free memory blocks.
- Limited available memory:
  - Even if there is a lot of free memory, fragmentation can prevent the allocation of larger blocks, which can lead to program errors.

SUMMARY

- Memory fragmentation is a serious problem that can negatively impact the performance and stability of programs. Understanding the causes of fragmentation and applying appropriate memory management techniques is key to creating efficient applications.
- To counteract the problem of memory fragmentation, a good programming practice is to use `realloc()` instead of `malloc()` or `alloc()` when working with the same data and needing to change its size.

Memory functions (`<string.h>`)

Used for working with **raw memory blocks**. Unlike `strcpy`, `strlen` and friends, they do **not** look for `'\0'` – they work with the number of bytes **you** give them.

Function	Description	Returns	Example
`memcpy(dest, src, n)`	Copies `n` bytes. Blocks **must not overlap**.	`dest`	`memcpy(buf2, buf1, n);`
`memmove(dest, src, n)`	Safe copy – works even if the blocks overlap.	`dest`	`memmove(a, b, 5);`
`memcmp(a, b, n)`	Compares `n` bytes.	`<0`, `0`, `>0`	`if (memcmp(a, b, 4) == 0)`
`memset(ptr, val, n)`	Fills `n` bytes with the byte value `val`.	`ptr`	`memset(buf, 0, 10);`
`memchr(ptr, val, n)`	Finds the first byte equal to `val` in the first `n` bytes.	pointer or `NULL`	`p = memchr(buf, 'x', 20);`

memcpy() vs memmove()

Inserting bytes into the middle of a buffer shifts the tail of the buffer **onto itself**. Source and destination overlap.

```
before:  a b c d e f . .        insert 2 bytes at index 2
                          move the tail "c d e f" two places right
after:   a b ? ? c d e f
```

```c
char s[16] = "abcdef";
memmove(s + 4, s + 2, 4);       /* "cdef" moves right by 2 -> "abcdcdef" */
memcpy(s + 2, "XY", 2);         /* fill the gap           -> "abXYcdef" */
```



- `memcpy` assumes the blocks do **not** overlap. If they do, the result is **undefined** – it may even work today and fail on another machine or compiler version.
- `memmove` always behaves as if it copied through a temporary buffer.
- If you are not sure whether the blocks overlap, use `memmove`.
- Removing bytes (shifting the tail **left**) also needs `memmove`.

`memset()`, `memcmp()` and `memchr()` 1/2

```c
unsigned char buf[8];
memset(buf, 0, sizeof buf);          /* all bytes -> 0x00 */
```



`memset` fills **bytes**, not elements:

```c
int a[3];
memset(a, 0, sizeof a);              /* OK: all zero              */
memset(a, 1, sizeof a);              /* every int is 0x01010101 = 16843009, not 1! */
```



`memset()`, `memcmp()` and `memchr()` 2/2

`memcmp` returns a negative number, zero or a positive number – only the **sign** is meaningful (it is not necessarily `-1` or `1`):

```c
if (memcmp(x, y, n) == 0) { /* equal */ }
if (memcmp("abc", "abd", 3) < 0) { /* "abc" comes before "abd" */ }
```



⚒ Do not use `memcmp` on `float` / `double` (different bit patterns can be equal, e.g. `0.0` and `-0.0`) or on structs (padding bytes are unpredictable).

`memchr` returns a **pointer** – subtract the start to get an index:

```c
const unsigned char *found = memchr(data, 'x', size);
if (found != NULL)
{
    int index = (int)(found - data);
}
```



Counting with `memchr()`

Search again **after** the previous hit:

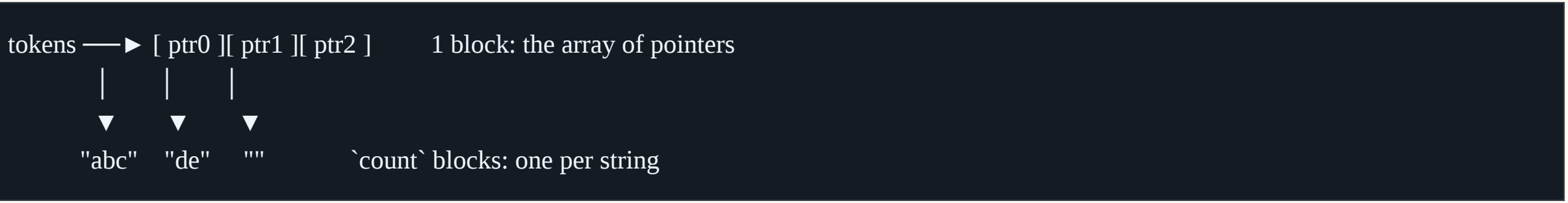
```c
int count_byte(const unsigned char *data, int size, unsigned char byte)
{
    int count = 0;
    const unsigned char *cursor = data;
    int remaining = size;

    while (remaining > 0)
    {
        const unsigned char *found = memchr(cursor, byte, remaining);
        if (found == NULL)
        {
            break;
        }
        count++;
        remaining -= (int)(found - cursor) + 1;
        cursor = found + 1;
    }
    return count;
}
```



Arrays of strings: `char **` 1/2

Two levels of allocation: **one** block of pointers and **one block per string**.




```c
char **tokens = malloc(3 * sizeof *tokens);
tokens[0] = malloc(4);
memcpy(tokens[0], "abc", 4);    /* 3 chars + '\0' */
tokens[1] = malloc(3);
memcpy(tokens[1], "de", 3);
tokens[2] = malloc(1);  tokens[2][0] = '\0';
```

Arrays of strings: `char **` 2/2

Freeing – **strings first, array last**:

```c
for (int i = 0; i < 3; i++)
{
    free(tokens[i]);
}
free(tokens);
tokens = NULL;
```



- If you `free(tokens)` first, you lose the only pointers to the strings: **3 leaked blocks**.
- A token needs `length + 1` bytes: do not forget room for the `'\0'`.
- If any allocation in the middle fails, free **everything allocated so far** before returning `NULL`.

Checklist: dynamic memory

- [ ] Every `malloc` / `calloc` / `realloc` result is checked for `NULL`.
- [ ] Sizes are written as `count * sizeof *pointer` (no magic numbers, no cast on `malloc`).
- [ ] Memory from `malloc` is written before it is read.
- [ ] `realloc` goes through a **temporary pointer**.
- [ ] Every block is freed **exactly once**, on **every** path out of the function.
- [ ] Pointers are set to `NULL` after `free()`.
- [ ] Nested objects are freed **inside first, outside last**.
- [ ] If a later allocation fails, earlier ones are freed.
- [ ] Never return the address of a local variable.
- [ ] `memmove` for overlapping blocks, `memcpy` otherwise; sizes are in **bytes**.
- [ ] The program was run with `-fsanitize=address`.

Summary

- The **stack** is managed automatically and is short-lived; the **heap** is managed by you and lives until you `free()` it.
- `malloc` gives uninitialized memory, `calloc` gives zeroed memory, `realloc` resizes (and may move) a block. Always check for `NULL`.
- `free` releases a block once. A pointer to freed memory is **dangling**: nullify it.
- A leak is a block you can no longer reach. The OS frees everything when the process ends, but a long-running program must not leak.
- `memcpy`, `memmove`, `memcmp`, `memset`, `memchr` work on raw bytes; use `memmove` when blocks overlap.
- Structures that own heap memory need clear ownership, deep copies and a matching destroy function.

THANK

You!